# World of Warships - Ship Performance Analysis

**Github Link:**

## 1. Introduction

In today's world of online multiplayer gaming, digital games have managed to integrate into our entertainment world, pulling in players from every part of the world. World of Warships stands out as a polished naval combat game, demanding strategy, teamwork, and a keen eye for performance. Given the extensive selection of ships, players frequently ponder which vessels truly excel on the battlefield.

However, as competition in such games is rising to new heights, issues related to balancing out ship performance come into play. Damage dealt, survival rates, and tactical contributions are essential factors that come into play while determining a ship's performance, but it's often difficult to understand how such factors are affecting overall performance as a whole. Hence, understanding ship performance data is essential to understand which ships are performing well and which are not.

### 1.1 Problem Formulation

This project extends beyond basic indicators such as damage output or win rate and utilizes a broader form of analysis for an examination of World of Warships through:

1. Comparing Ship Performance Across Tiers, Classes, and Nations
2. Predicting Win Rate from Multiple Performance Metrics

**Research Question:**
How do ship performance characteristics differs with tiers, classes, and nations, and can we predict win rate accurately using other performance variables?

### 1.2 Data Acquisition

- **Origin**: Publicly available data set obtained from Kaggle, a data science competition platform, under the title 'World of WarShips: Ship Performance Stats'.
- **Content**: Performance statistics related to 925 ships at various tiers (I to XI), classes (BB, CA, DD, CV, SS), and nations.
- **Variables**: 16 columns containing ship names, tiers, classes, nations, player counts, battles played, as well as performance measures such as damage, frags, base XP, win rate, capture, defense, spotting, potential damage, aircraft kills, survival rate, etc.

| | Ship | Tier | Class | Nation | Players | Battles | Base XP | Damage | Frags | Win rate | Capture | Defence | Spotting | Potential | Aircraft | Survival rate |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | Patrie | 11 | BB | France | 9092 | 337170 | 1388 | 125865 | 0.936 | 0.512 | 2.13 | 4.74 | 19358 | 1809099 | 5.60 | 0.426 |
| 1 | Hannover | 11 | BB | Germany | 16503 | 319701 | 1150 | 86471 | 0.633 | 0.485 | 1.81 | 4.16 | 20080 | 1930510 | 6.42 | 0.240 |
| 2 | Satsuma | 11 | BB | Japan | 40417 | 1562549 | 1246 | 112667 | 0.809 | 0.499 | 1.32 | 3.21 | 17402 | 1730502 | 4.15 | 0.469 |
| 3 | Devastation | 11 | BB | U.K. | 5660 | 140172 | 1323 | 127807 | 0.766 | 0.499 | 1.98 | 4.78 | 20776 | 1896948 | 4.71 | 0.499 |
| 4 | Maine | 11 | BB | U.S.A. | 10744 | 352970 | 1344 | 114812 | 0.890 | 0.513 | 2.32 | 4.11 | 22194 | 2078203 | 7.70 | 0.445 |

*Figure 1: World of Warships ship performance dataset*

## 2 Data Analysis

### 2.1 Clean and preprocess data

1. **Remove Unnecessary Columns**: No irrelevant or unnamed columns were present in the data, so nothing is to be removed in this case.
2. **Standardize Text Fields**: The ship names, class names, and nation names were cleaned to remove any extra spaces and make them consistent.
3. **Convert Columns to Numeric Type**: All the columns related to the performance of the ship, like Damage, win rate, and Base XP, were changed to a numeric data type for precise analysis and calculations.
4. **Handle Missing Values**: Non-numeric data or missing data is replaced with the correct null and then replaced with 0, maintaining the consistency of the data for modeling.
5. **Remove Duplicates**: The data is checked for any duplicate rows, and since none were found, each row is unique and represents a different ship.

After cleaning, the dataset contained **925 unique rows and 16 columns**.

### 2.2 Exploratory Data Analysis

1. **Win Rate Distribution**: The win rate distribution is roughly normal, with a peak around 50–52%, showing most ships win about half their battles.

   - Mean Win Rate: 51.7%
   - Median: 51.3%
   - Range: 0.00 → 72%

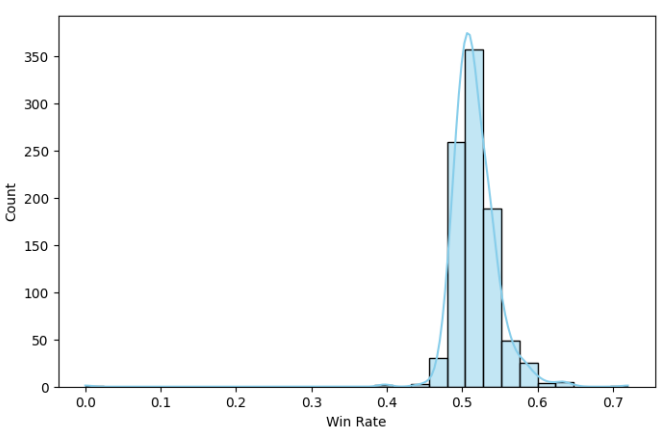


*Figure 2: Distribution of Win Rate*

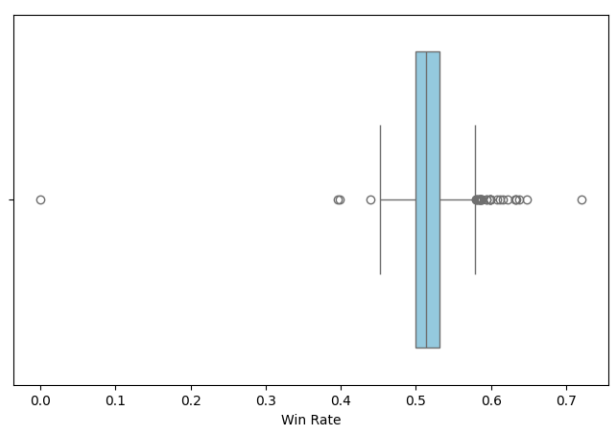


*Figure 3: Box Plot of Win Rate*

## 2. Damage Distribution

- Average damage per match varies widely across ships.
- Mean Damage: 49,264
- Range: 9,487 → 145,521

Tiers and class strongly influence damage output. Which means, higher-tier ships tend to have higher damage.

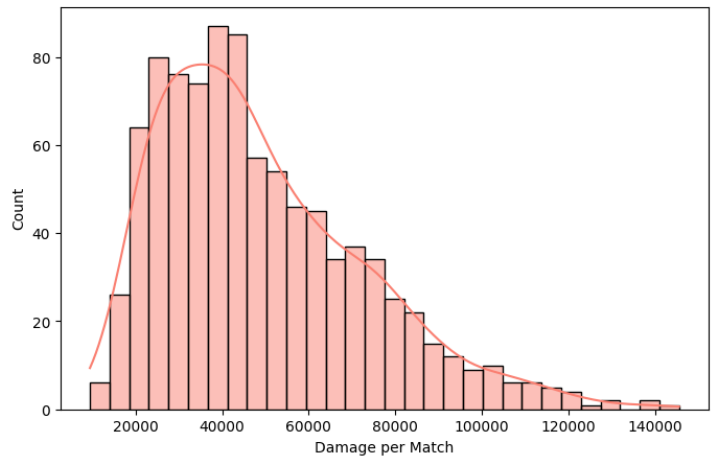


*Figure 4: Histogram of Damage*

## 3. Tier-wise Summary

Ship counts by Tier:

- Tier I: 13
- Tier II: 29
- Tier III: 39
- Tier IV: 43
- Tier V: 71
- Tier VI: 97
- Tier VII: 112
- Tier VIII: 185
- Tier IX: 153
- Tier X: 162
- Tier XI: 21

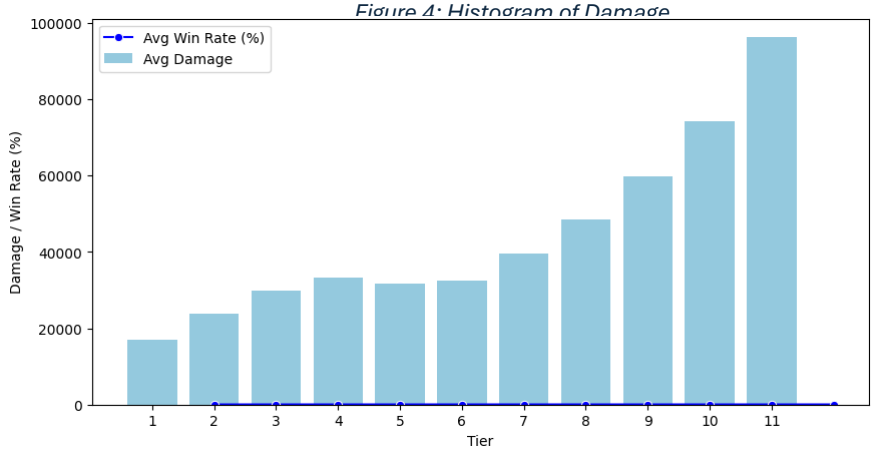


*Figure 5: Average Damage and Win Rate by Tier*

Most ships are mid-to-high tier (7 - 10), with Tier VIII being the most common.

## 4. Class-wise Breakdown

Ship counts by class:

- CA (Cruiser): 351
- BB (Battleship): 250
- DD (Destroyer): 226
- CV (Carrier): 69
- SS (Submarine): 29

The cruisers and battleships are the majority, while the carriers and submarines are the minority.

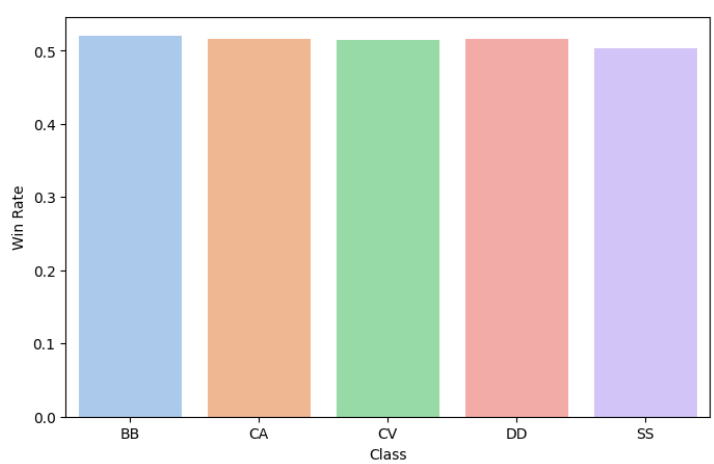


*Figure 6: Average Win Rate by Class*

## 5. Nation Overview

Top 5 nations by ship count:

- U.S.A.: 159
- Japan: 142
- Germany: 118
- U.K.: 104
- U.S.S.R.: 102

Dataset covers major nations well, allowing meaningful cross-national comparisons.

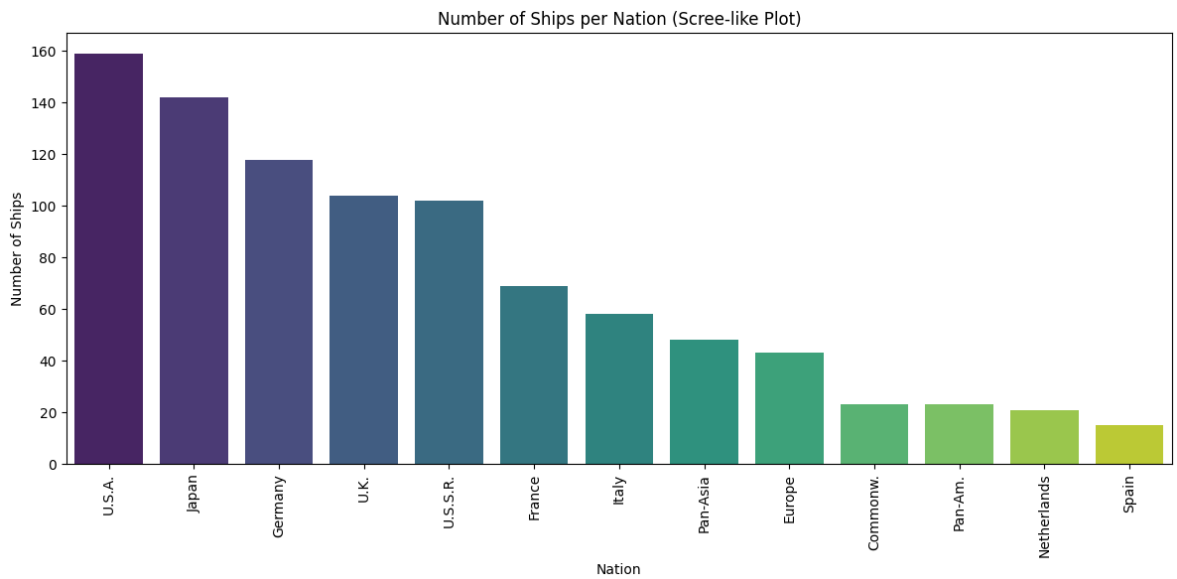


*Figure 7: Number of Ships per Nation*

## 6. Correlation Analysis

Pearson correlation of Win Rate with key numeric features:

- Survival Rate: +0.25 (strongest correlation)
- Base XP: +0.20
- Damage: +0.12
- Capture: +0.10
- Frags: +0.04
- Defense: +0.01

Survival and Base XP are better predictors of winning than raw damage, emphasizing tactics and objectives over sheer firepower.

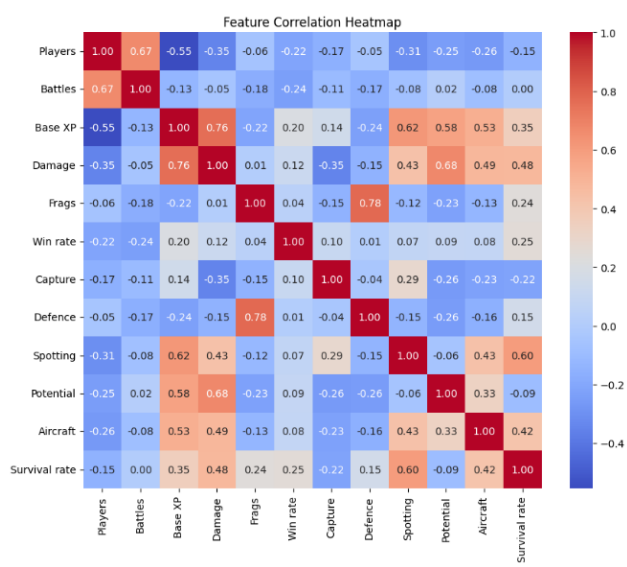


*Figure 8: Feature Correlation Heatmap*

## 7. Principal Component Analysis (PCA)

Features used: Damage, Frags, Base XP, Capture, Defense, Spotting, Potential, Aircraft, Survival Rate

Explained Variance (First 3 PCs):

- PC1: 37.6%
- PC2: 22.2%
- PC3: 17.4%
- Cumulative: 77.2%

PC1 is dominated by Base XP, Damage, and Aircraft (offensive/tactical metrics).

Most variation in ship performance can be captured using 3 principal components → simplifies analysis without losing important information.

| | PC1 | PC2 | PC3 |
|---|---|---|---|
| Damage | 0.475536 | 0.120655 | -0.229115 |
| Frags | -0.140899 | 0.608421 | -0.110273 |
| Base XP | 0.481665 | -0.045079 | 0.112776 |
| Capture | -0.086476 | -0.193150 | 0.615154 |
| Defence | -0.188381 | 0.564317 | -0.048410 |
| Spotting | 0.358215 | 0.124211 | 0.529195 |
| Potential | 0.328866 | -0.201008 | -0.471318 |
| Aircraft | 0.394222 | 0.077910 | -0.024734 |
| Survival rate | 0.295237 | 0.442233 | 0.197661 |

*Figure 9: PC Loadings*

## 3 ML/AI Methods

The goal of the project is to predict the win rate of a ship using the available performance metrics. Since the win rate is a continuous number between 0 and 1, this is a regression problem.

All the chosen feature variables are continuous: Damage, Frags, Base XP, Capture, Defense, Spotting, Potential, Aircraft, Survival rate. To test how well the model generalizes, the data is split randomly into 70% training data and 30% testing data.

```
y = df['Win rate']
X = df[features_for_pca]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

*Figure 10: Feature Selection*

### 3.1 Linear Regression

We used a simple linear regression as our baseline. This is simple and easy to understand, and we can easily see the impact of any feature on the win rate. When we ran this model on our test data set, it failed to deliver. The $R^2$ value for this model was merely 0.066, which means that this linear mix of features is

able to explain only a small fraction of the total variability in the win rate. The MAE for this model was approximately 0.021, meaning that the predictions were off by about 2%. The value for MSE turned out to be 0.000736. Among the features, Survival rate had the most significant positive impact on the win rate. Base XP and Capture came in second and third in this regard. However, there is likely a high degree of multicollinearity in the features, making this coefficient value unreliable.

```
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

*Figure 11: Linear Regression Model*

### 3.2 Random Forest Regressor

Next, a Random Forest Regressor with 100 trees was employed to account for non-linear effects as well as interactions between the features. It performed better than the linear regression model with a comfortable margin, yielding an $R^2$ score of 0.608, which implies that about 61% of the variance in the win rate is explained. The MAE was reduced to 0.012, while the MSE was reduced to 0.000309, indicating a more accurate prediction model. In terms of feature importance, the top factors influencing the win rate were Frags, Defence, and Survival rate. This is intuitive, as the longer the survival rate, the higher the chances of winning the game. Damage dealt and Base XP were the secondary factors, implying that while dealing damage is important, surviving the entire duration of the game and contributing to the team's objectives is vital in the game.

```
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

*Figure 12: Random Forest Regressor*

### 3.3 Neural Network Model

A compact neural network model was designed with three dense layers having 128, 64, and 32 units, and a dropout rate of 0.2 after the second and third layers to help in modeling any potential nonlinear relationship between the features and win rate. The training process involved 100 epochs with a batch size of 16 and the Adam optimizer. The $R^2$ value for this model was 0.075, MAE was 0.020, and MSE was 0.000729. Even though it performed better than the linear regression model by a small margin, it still failed to achieve better results than those obtained by the Random Forest model, which proves that the relationship between the ship's performance metrics cannot be modeled by a simple neural network model.

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
history = model.fit(X_train, y_train, epochs=100, batch_size=16, validation_split=0.2, verbose=0)
```

*Figure 13: Neural Network Model*

### 3.4 Model Comparison

When comparing all of these models, it is evident that the Random Forest performed much better than both the Linear Regression and Neural Network models. This is because, out of all these models, it had the lowest prediction error, meaning that it was able to explain the most variance of the win rate. Therefore, this is the best method to use with this data set. While these Linear Regression and Neural Network models were helpful, they were not able to account for all variables affecting ships. This shows that survivability, objective play (capture/defense), and kills are much more indicative of a ship's success, not damage.

Figures 7, 8, and 9 show the predicted vs. actual win rates using Linear Regression, Random Forest, and Neural Network, respectively.

## 4 Reflection

### Challenges faced

- **Multicollinearity:** There is a relationship between the predictors Damage, Base XP, Potential, and Spotting. This affects linear model coefficients interpretability.
- **Feature engineering choices:** Had to decide whether to include Battles and/or Players as predictors, which are not inherent properties of a ship and may impact model fairness.
- **Heterogeneous roles:** Different ship classes (BB vs DD vs CV) have different purposes. A unified model must account for class-specific behavior or be stratified by class for more precise predictions.

### Key learnings

- Survival and objective play (capture/defense) are strong contributors to win rate, not just raw damage.
- Tree-based ensemble models, like Random Forest, are suitable for this domain, as they can handle the interaction between kills, defense, and survival.
- Dimensionality reduction, like PCA, is effective as the top 3 principal components account for ≈77% variance, so you can reduce the complexity of the features for clustering, etc.

## 6 References

Dataset: *World of WarShips — Ship Performance Stats*, Kaggle — wows_ship_stats.csv.

https://www.kaggle.com/datasets/mjawad17/world-of-warships-ship-performance-stats?resource=download

Scikit-learn documentation (PCA, RandomForest, metrics).

General ML references: Géron, Hands-On Machine Learning (recommended if you need to justify choices).